BenefitPulse Engine — System Design Report

System Design Report | Standalone PDF edition

Document type: High-level system design Product: BenefitPulse Engine (Card Benefit Activation Prototype) Version: 1.0 · July 2026

1. Executive summary

BenefitPulse is a full-stack prototype that:

1. Observes posted card charges,
2. Runs a multi-agent coverage assessment (LLM + rules + RAG),
3. Records outcomes as claim-eligible or outside coverage, and
4. Prefills a claim packet when coverage exists, with a policy assistant for Q&A.

It is designed for local execution of UI and API processes, with cloud free-tier services for intelligence (Google Gemini) and operational data (Supabase Postgres + Auth). Vector search runs on disk-local ChromaDB.

The product deliberately surfaces negative outcomes (outside coverage) with the same seriousness as positive ones, so users and reviewers can trust the system's discrimination—not only its hits.

2. Problem statement

Card members often leave value on the table because:

- Benefit terms are long and fragmented across PDFs.
- Eligibility depends on category, timing windows, and exclusions.
- Filing a claim requires gathering fields and documents under stress.

Opportunity: Automatically map recent spend to likely protections, score confidence, prefill claims, and answer policy questions—with transparent "no" answers when policy excludes the charge.

3. Goals and non-goals

3.1 Goals

ID	Goal
G1	Detect candidate protections from transaction signals
G2	Ground reasoning in a versionable policy knowledge base (RAG)
G3	Persist both eligible and outside-coverage assessments
G4	Prefill claim packets and support document upload
G5	Provide conversational, policy-aware assistance

ID	Goal
G6	Run only with real Supabase + Gemini credentials (no fake offline store)

3.2 Non-goals (prototype)

ID	Non-goal
N1	Official issuer integration or regulated claims decisioning
N2	Real-time bank network ingestion (prototype uses seed/simulate/inject)
N3	Multi-tenant enterprise RBAC / SOC2 certification
N4	Guaranteed approval of any claim

4. Stakeholders and users

Stakeholder	Interest
Card member (end user)	Find coverage, file faster, understand exclusions
Demo reviewer	End-to-end credible fintech story
Engineer	Clear modules, env, and extension points
Operator	Seed, keys, reindex, health checks

5. High-level architecture

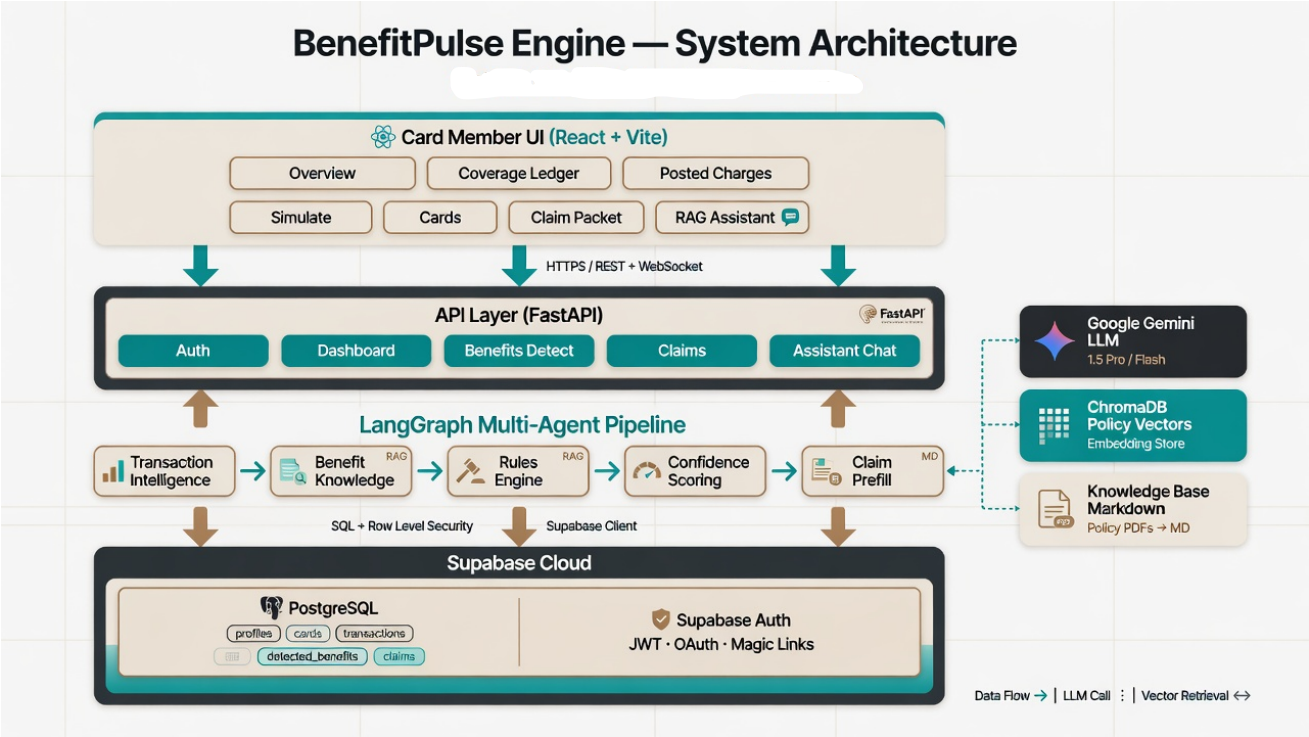

Figure 1 — BenefitPulse layered architecture.

5.1 Logical layers

```
┌──────────────────────────────────────────────────────────┐
│  Presentation: React SPA (taskbar workspace)             │
├──────────────────────────────────────────────────────────┤
│  API: FastAPI (auth, dashboard, detect, claims, chat)    │
├──────────────────────────────────────────────────────────┤
│  Intelligence: LangGraph agents + Gemini + ChromaDB RAG  │
├──────────────────────────────────────────────────────────┤
│  Persistence: Supabase Postgres + Auth                   │
│  Vectors: ChromaDB on local disk                         │
└──────────────────────────────────────────────────────────┘
```

5.2 Trust boundaries

Boundary	Trust decision
Browser ↔ API	JWT bearer from Supabase Auth
API ↔ Postgres	Service role for agents; RLS for user paths
API ↔ Gemini	Outbound HTTPS with API key (server only)
API ↔ Chroma	Local process / filesystem

6. Component design

6.1 Frontend components

Component	Responsibility
Layout / taskbar	Navigation, session chrome, assistant host
Overview	KPIs + recent assessments (eligible + outside)

Component	Responsibility
Coverage ledger	Full detection list
Posted charges	Assessment trigger + outcome highlighting
Simulate	Live pipeline demo without cluttering Overview
Claim packet	Prefill edit, uploads, submit
Assistant chat	RAG Q&A with optional claim context

6.2 Backend components

Component	Responsibility
`config.Settings`	Env load + fail-fast validation
`store` facade	Supabase-only operational store
`supabase_store`	CRUD + `apply_pipeline_result`
`pipeline`	Multi-agent detection graph
`assistant`	Chat with retrieval + optional fallback text
`vector_store`	Chunk, embed, query, reindex
Routers	Thin HTTP adapters over services

6.3 External services

Service	Role	Failure mode
Supabase Auth	Identity	Login/signup fail closed
Supabase Postgres	System of record	API errors to client
Gemini	LLM + embeddings	Step may use heuristics; server still requires key at boot
ChromaDB	Vector index	May fall back to weaker retrieval if misconfigured

7. Key workflows

7.1 Coverage assessment sequence

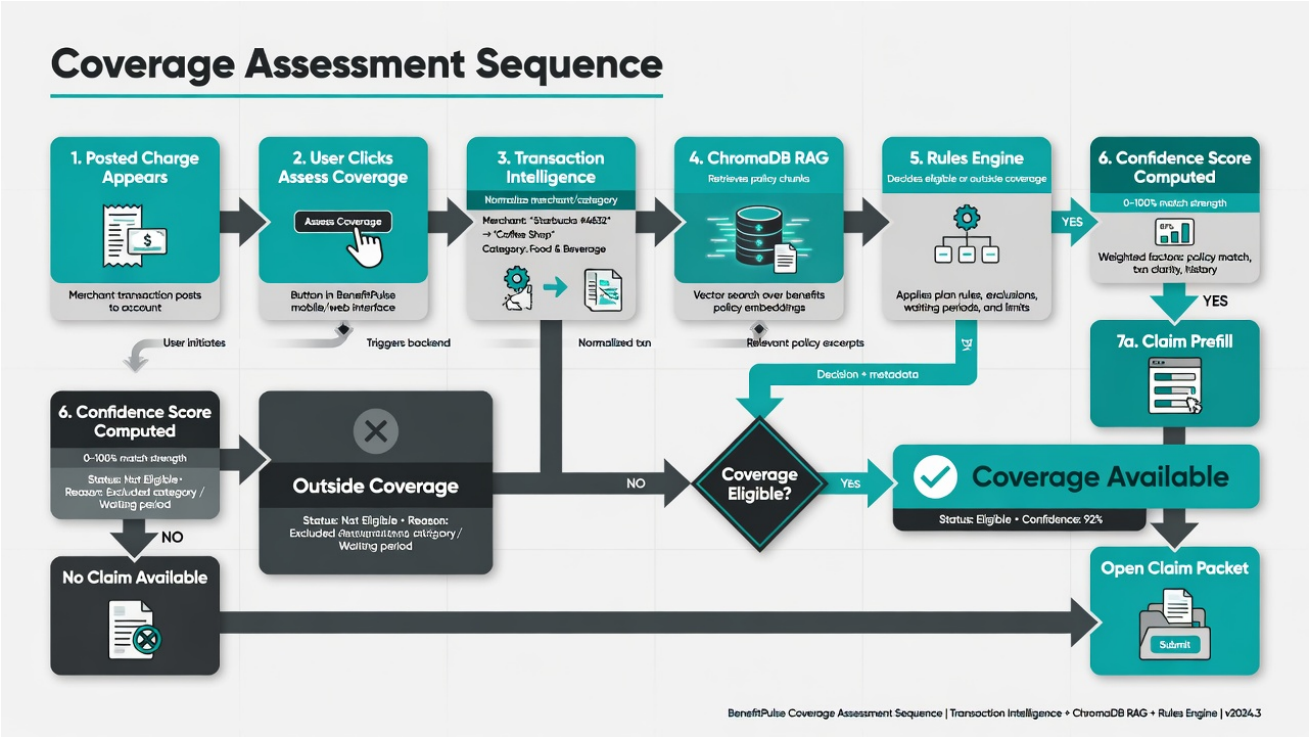

Figure 2 — Assessment sequence from charge to outcome.

Happy path (eligible):

1. Charge exists in `transactions`.
2. User (or inject) calls `POST /api/benefits/detect/{id}`.
3. Intelligence → RAG → Rules → Confidence → Prefill.
4. `detected_benefits` row with claim-eligible status.
5. Draft `claims` row created.
6. UI shows Coverage available / Claim ready.

Negative path (outside coverage):

1. Same pipeline entry.
2. Rules mark ineligible (e.g. Food & Dining exclusion).
3. `detected_benefits` row with `not_eligible` / "Outside coverage".
4. No claim draft.
5. UI highlights row as outside coverage; claim CTA disabled.

7.2 Authentication flow

1. Signup/login via Supabase Auth API (anon client).
2. Access token returned to SPA.
3. SPA stores token; sends `Authorization: Bearer ...`.
4. API validates via store / JWT secret.
5. Signup seeds starter wallet cards.

7.3 Claim filing

1. Open claim for eligible benefit.
2. Edit prefilled fields.

3. Upload documents → update missing list.

4. Submit → claim status transitions; dashboard stats update.

8. Data architecture

8.1 Conceptual ER (simplified)

```
auth.users 1■■1 profiles
profiles 1■■* cards
profiles 1■■* transactions
transactions 1■■0..1 detected_benefits   (latest assessment model)
detected_benefits 1■■0..1 claims          (only if claim-eligible)
claims 1■■* documents
profiles 1■■* agent_runs                  (audit)
```

8.2 Design choices

Choice	Rationale
Persist outside-coverage outcomes	Enables transparent ledger & training UX
Local Chroma + cloud Postgres	Free-tier friendly; policies stay versionable in git
Service role for pipeline	Agents must write across tables without user JWT constraints
Markdown knowledge base	Easy edits; reindex = new embeddings

9. Intelligence design

9.1 Hybrid decisioning

```
        ■■■■■■■■■■■■■■■■
        ■  Gemini LLM ■  soft judgment / wording / intel
        ■■■■■■■■■■■■■■■■
               ■
Transaction → Intel → RAG candidates → Rules (hard gates) → Confidence → Prefill?
               ■              ■
               ■              ■■ Chroma policy vectors
               ■■ Heuristic fallbacks if a Gemini call fails mid-run
```

Principle: Soft models propose and explain; rules enforce eligibility. This reduces silent over-claims.

9.2 Confidence

Composite score from signals such as merchant match, category fit, policy match, and window remaining. UI maps:

- High ≥ 0.85
- Medium ≥ 0.60
- Low < 0.60

Low confidence still may be claim-eligible (e.g. near end of window) but is flagged for human caution.

9.3 RAG

- Corpus: six markdown policy docs.
- Embeddings: Gemini embedding model.
- Store: Chroma collection under `backend/data/chroma`.

• Consumers: detection knowledge node + chat assistant.

10. UX design & wireframes

10.1 Information architecture

Taskbar-first navigation reduces the “stuffed dashboard” problem: each tool owns a route.

```
Overview ■■ snapshot
Coverage ■■ full ledger
Charges ■■ assess / re-assess
Simulate ■■ live agents
Cards / Profile ■■ wallet & account
```

10.2 Wireframes

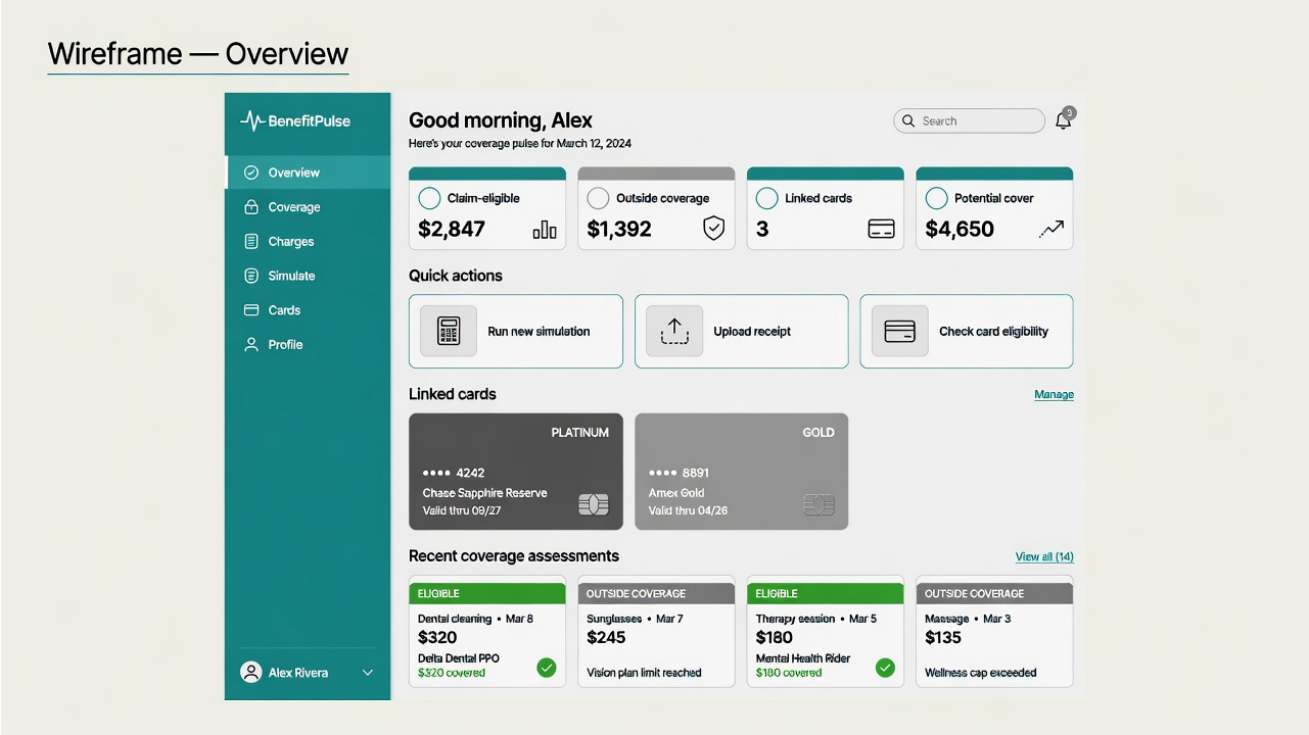

Figure 3 — Overview wireframe: KPIs, shortcuts, cards, recent assessments.

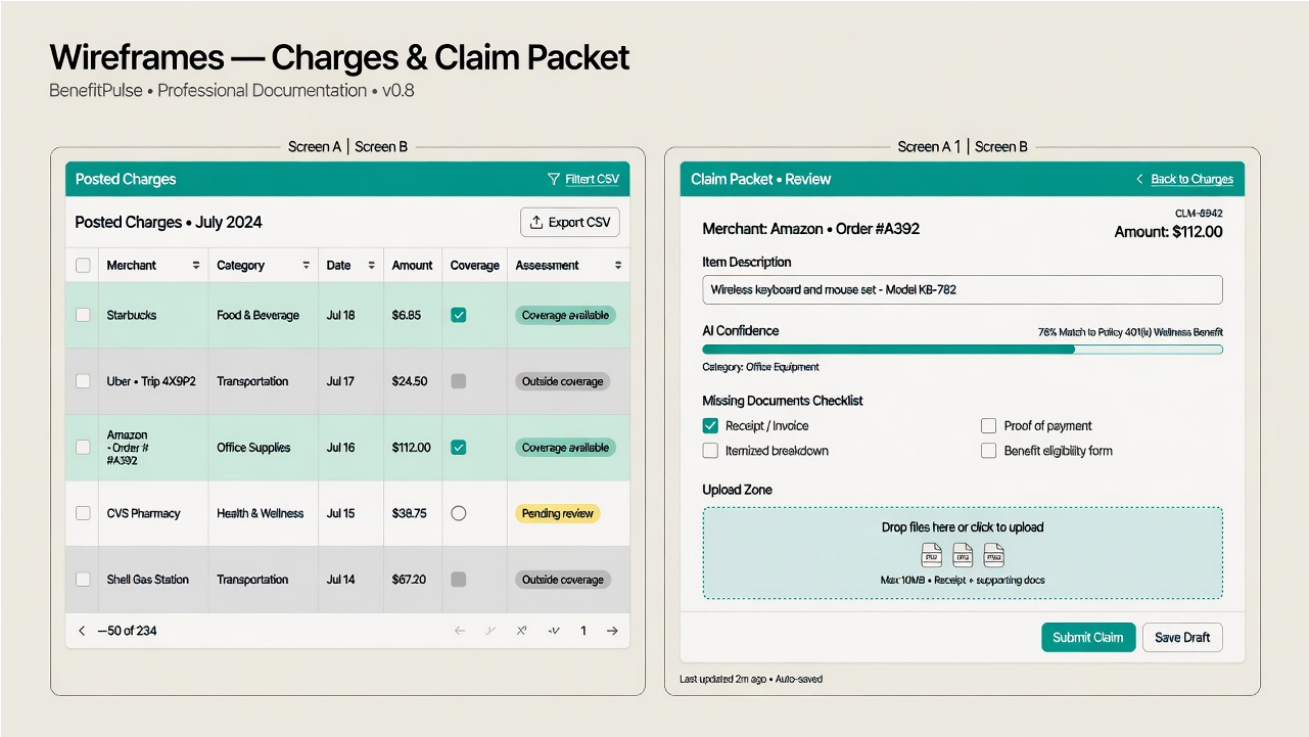

Figure 4 — Posted charges outcomes + claim packet structure.

10.3 Visual language

Token	Usage
Cream background	Calm, non “AI-blue” full-page wash
Teal primary	Trust / primary CTAs
Bronze accents	Secondary emphasis
Emerald highlight	Claim-eligible
Stone/grey highlight	Outside coverage
Multi-color card faces	Platinum slate / Gold / Green / Blue Cash variants

10.4 Finance microcopy standards

Prefer	Avoid
Coverage assessment	“AI magic detect”
Claim-eligible / Outside coverage	“Good / bad AI guess”
Claim packet	“Form dump”
Potential cover	Vague “savings” without context

11. Cross-cutting concerns

11.1 Security

- Secrets only in server env.
- JWT validation for protected routes.
- RLS as defense in depth.
- No full card numbers in UI/API payloads.

11.2 Reliability

- Fail-fast boot if credentials missing (no silent demo fallback).
- Pipeline can degrade a step if Gemini rate-limits, but credentials remain mandatory.
- Seed script is idempotent for the sample member (clears and reloads scenarios).

11.3 Scalability (prototype horizon)

Load	Approach
Demo / single user	Current monolith + free tier is sufficient
Many concurrent detects	Queue jobs; rate-limit Gemini; scale API workers
Large policy corpus	Chunking strategy + collection partitioning

11.4 Observability

- Structured startup logs.
- `/health` and `/api/system/status` for architecture checklist.
- Optional `agent_runs` audit table.

12. Risks and mitigations

Risk	Impact	Mitigation
Hallucinated eligibility	Wrong claim hope	Hard rules gate; show outside coverage explicitly
Gemini quota	Degraded intel	Fallback models list; rules still decide
Stale Chroma index	Wrong citations	Reindex script + status `chunk_count`
Email confirmation friction	Demo login fail	Seed + Auth settings guidance
Mistaken for official Amex	Compliance / trust	Disclaimers in UI/docs

13. Alternatives considered

Alternative	Why not (for this prototype)
Fully offline JSON demo store	Undermined “real stack” story; removed
Single monolithic LLM call	Harder to audit; weaker rules separation

Alternative	Why not (for this prototype)
Hosted vector DB only	Extra cost; local Chroma is enough for demo
Dark navy “AI SaaS” aesthetic	Overused; reduced with cream/teal/bronze system

14. Future roadmap (suggested)

1. Issuer feed adapter — Replace simulate/inject with secure transaction import.
2. Human-in-the-loop review — Queue low-confidence claim-eligible items.
3. Document OCR — Extract receipt fields into prefill.
4. Multi-card optimization — Suggest which linked product best matches a category.
5. Hardened production deploy — Separate front/back, stricter CORS, secret manager, monitoring.

15. Conclusion

BenefitPulse demonstrates a credible, end-to-end card benefit activation architecture:

- Cloud identity and data (Supabase)
- Grounded intelligence (Gemini + ChromaDB RAG + rules)
- Transparent outcomes (claim-eligible and outside coverage)
- Member-facing workflow (assess → packet → submit → assistant)

The design favors clarity and trust over “always yes” automation—essential for any fintech-adjacent prototype that discusses claims and coverage.